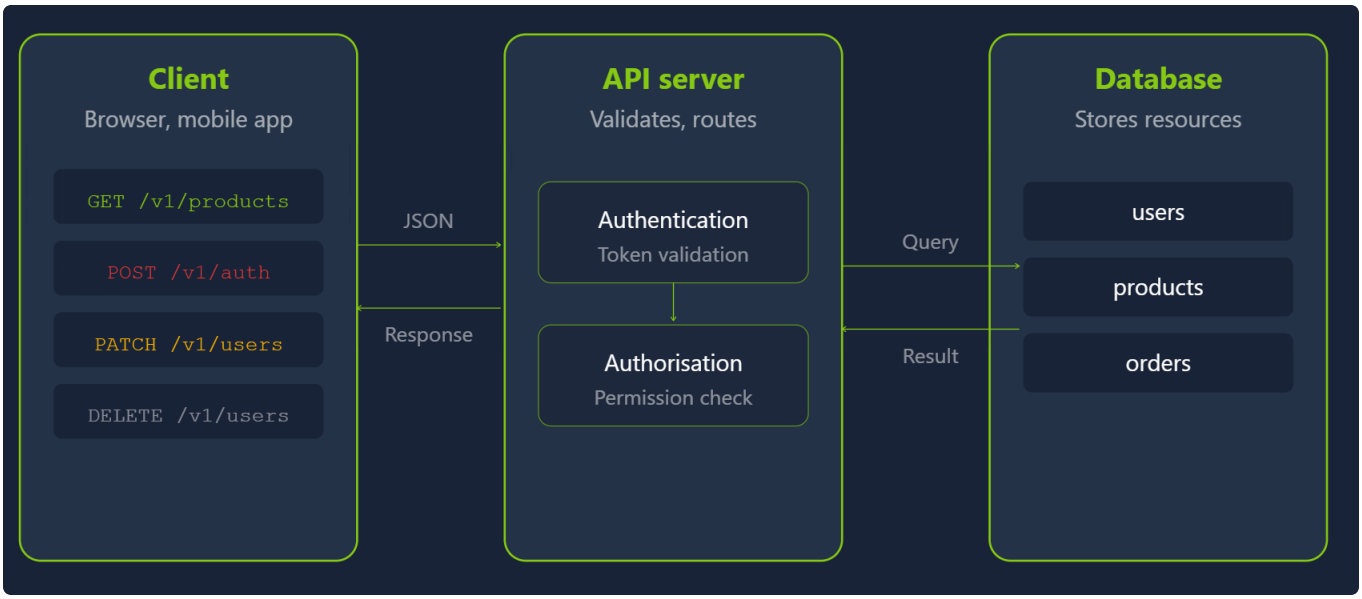
API Pentesting

How REST API's Work

It is an architectural style, a set of conventions that developers follow when building APIs over HTTP. An API that follows these conventions is called a **RESTful API**.



Resources and Endpoints

A resource is any object or piece of data the API exposes, such as a user, a product, or an order. Each resource is identified by a URL, referred to as an **endpoint**. For example, an e-commerce API might expose endpoints like `/v1/users`, `/v1/products`, and `/v1/orders`

HTTP Methods

Method	CRUD Operation	Description	Example
`GET`	Read	Retrieves a resource. Should never modify data on the server.	`GET /v1/products/1` returns the data for product 1.
`POST`	Create	Creates a new resource or submits data for processing.	`POST /v1/auth/login` with a JSON body authenticates the user and returns a token.
`PUT`	Full Update	Replaces an existing resource entirely. Omitted fields may be set to null or a default value.	`PUT /v1/users/4` replaces all of user 4's data with the provided body.
`PATCH`	Partial Update	Modifies only the fields included in the request body. Everything else remains unchanged.	`PATCH /v1/users/me` updates only the specified fields.
`DELETE`	Delete	Removes a resource from the server.	`DELETE /v1/users/4` deletes user 4.

A `PUT` request expects the full object, while a `PATCH` request expects only the fields that need changing

Status Codes

Every API response includes an HTTP status code that indicates what happened with the request. The most relevant codes for API security testing are:

Code	Meaning	Security Relevance
`200`	OK	Request succeeded
`201`	Created	New resource created (typically after `POST`)
`204`	No Content	Success with no response body (typical for `DELETE`)
`400`	Bad Request	Malformed request; useful for understanding input validation
`401`	Unauthorized	Authentication is missing or invalid
`403`	Forbidden	Authenticated but not authorised; key indicator during BOLA testing
`404`	Not Found	Resource does not exist
`405`	Method Not Allowed	HTTP method not supported for this endpoint
`429`	Too Many Requests	Rate limiting in effect
`500`	Internal Server Error	Server-side failure; may indicate injection or logic flaws

A `401` means the server does not know who you are. A `403` means the server knows who you are but you are not permitted to access the requested resource.

Request and Response Structure

A typical API request consists of the HTTP method, the endpoint URL, headers (carrying metadata such as authentication tokens and content type), and optionally a body (carrying the data payload)

Authentication Mechanisms

API Keys

This is the simplest form of authentication. The client includes a static key in a header (commonly `X-API-Key`). The server looks up the key to identify the client. However, API keys are long-lived, often shared across environments, and if leaked, grant full access until rotated.

Bearer Tokens

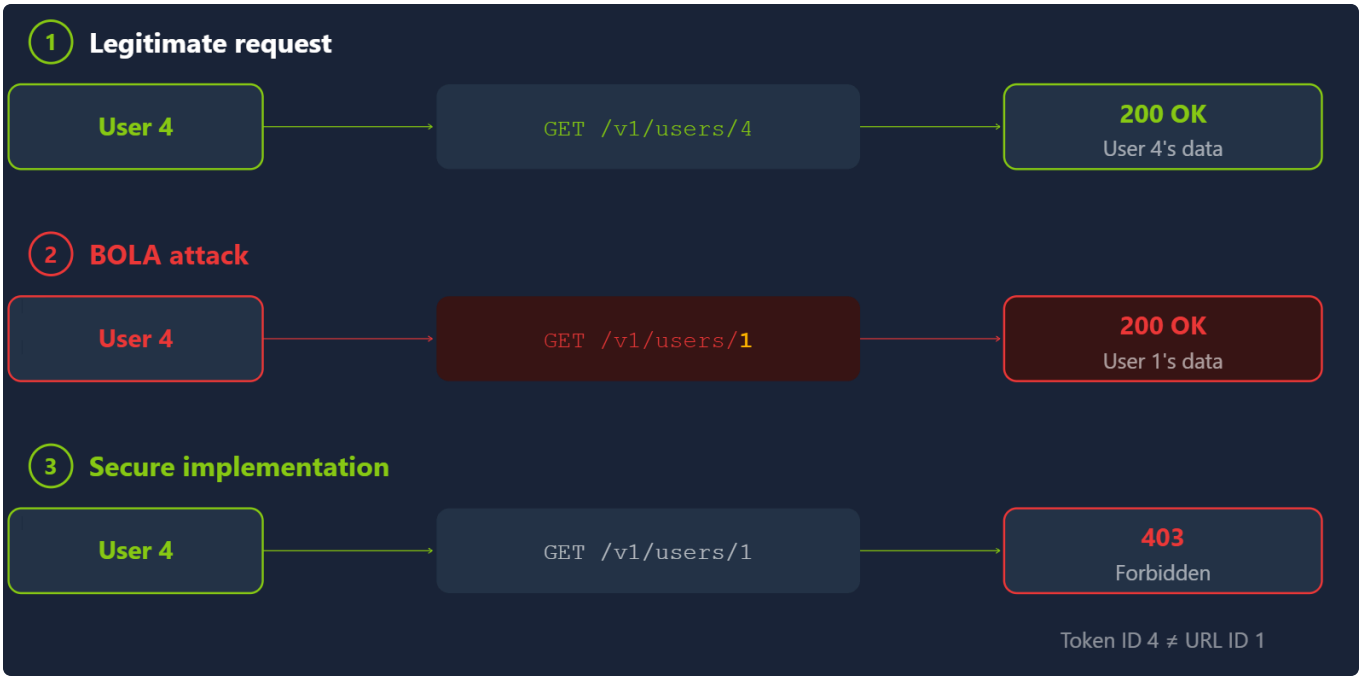
In this authentication method, the client first authenticates by submitting credentials to a login endpoint, and the server returns a token. The client includes this token in the `Authorization` header of subsequent requests. These tokens are typically short-lived and can be revoked.

JSON Web Tokens (JWTs)

This is the most common format for bearer tokens. A JWT consists of three Base64-encoded parts separated by dots: a header (specifying the algorithm), a payload (containing claims such as user ID, role, and expiration), and a signature (ensuring the token has not been tampered with). The payload is not encrypted, only encoded. Anyone with the token can read its contents.

Broken Object Level Authorisation (BOLA)

is an authorisation failure that occurs when an API returns a requested object without verifying whether the requesting user is permitted to access it



Why BOLA Is the Number One API Risk

BOLA holds the top position in the OWASP API Security Top 10(opens in new tab) because it appears in a large number of real-world APIs and frequently leads to mass data exposure.

RESTful APIs reference objects through predictable, structured URLs. A request to `GET /v1/orders/1045` retrieves order 1045. The identifier is directly in the URL and is usually a sequential integer. If the API does not verify ownership before returning the object, any authenticated user can access any other user's data by changing the number.

How BOLA Works

Consider the following scenario. You are authenticated as user ID 4 (testuser) and request your own order history by sending `GET /v1/users/4/orders`. The server sees a valid token, confirms user 4 exists, and returns your orders. However, if you change the user ID in the URL to `GET /v1/users/1/orders`, you are still authenticating as user 4 but requesting user 1's data.

In a properly secured API, the server would compare the token's user ID against the URL's user ID and return `403 Forbidden`

BOLA is not limited to URL path parameters. It can appear in query strings (`?owner_id=13`), request body fields (`{"user_id": 13}`), and custom headers (`X-User-ID: 13`).

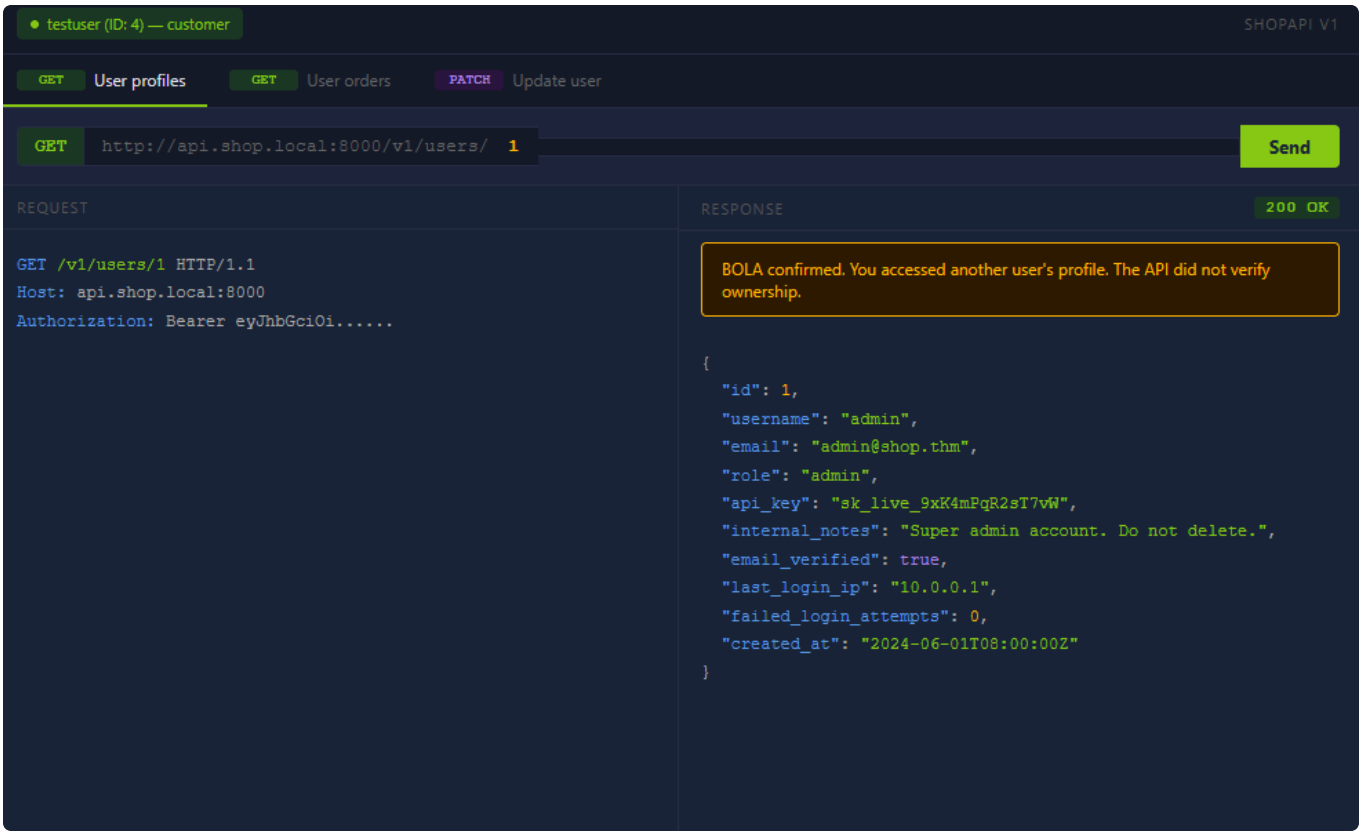
Scaling the Attack

Once an attacker confirms a vulnerable endpoint, iterating through all possible object IDs to extract data for every user in the system is trivial. If user IDs are sequential integers, a simple loop from 1 to 1000 pulls the data for every user in seconds. This is why BOLA is not just a theoretical concern. It enables mass data exfiltration with minimal effort.

Some APIs use UUIDs instead of sequential integers. This adds obscurity but is not a security control. UUIDs can be leaked through other endpoints, API responses, or error messages.

Practical Exercise

my own profile is user 4 I changed and accessed user 1s details and it didnt show a 403 error meaning that BOLA vulnerability is confirmed



GET User profiles | GET User orders | PATCH Update user

GET http://api.shop.local:8000/v1/users/ 1 /orders Send

REQUEST

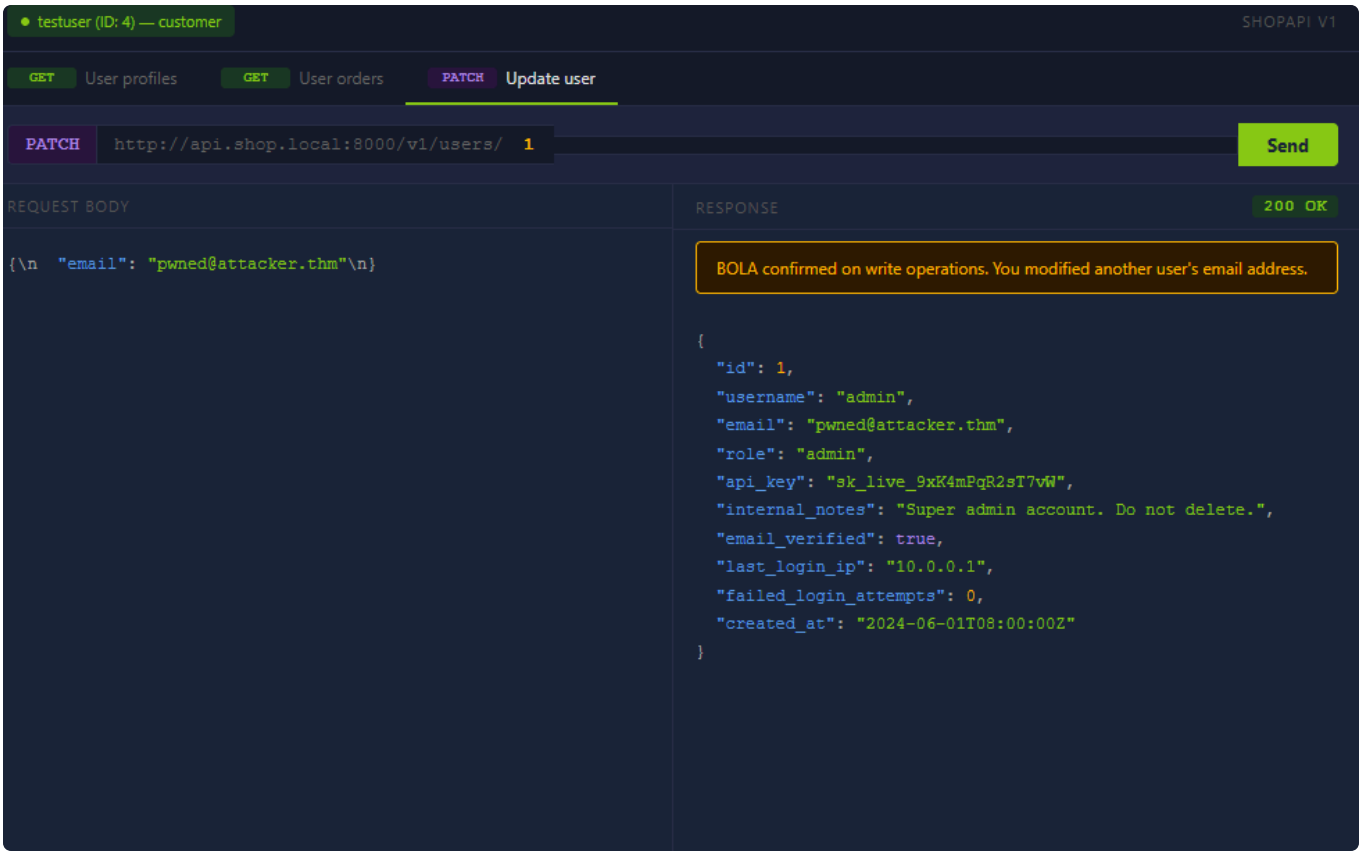
```
GET /v1/users/1/orders HTTP/1.1
Host: api.shop.local:8000
Authorization: Bearer eyJhbGciOi......
```

RESPONSE 200 OK

BOLA confirmed. You accessed another user's order history.

Flag found in the response data.


```
{
  "orders": [
    {
      "id": 1,
      "user_id": 1,
      "items": [
        {
          "product_id": 1,
          "qty": 2
        }
      ],
      "total": 99.98,
      "status": "completed",
      "shipping_address": "123 Admin St, London"
    },
    {
      "id": 2,
      "user_id": 1,
      "items": [
        {
          "product_id": 3,
          "qty": 1
        }
      ],
      "total": 149.99,
      "status": "shipped",
      "shipping_address": "123 Admin St, London",
      "flag": "THM{b0la_1s_the_numb3r_0ne_ap1_r1sk}"
    }
  ]
}
```



Broken Authentication and Excessive Data Exposure

Lack of Rate Limiting on Login Endpoints

Broken Authentication is a broad category of flaws in how an API verifies a user's identity. One of the most common manifestations is a login endpoint with no rate limiting, allowing an attacker to submit thousands of credential combinations per minute

Without rate limiting, brute-force and credential-stuffing attacks become trivial.

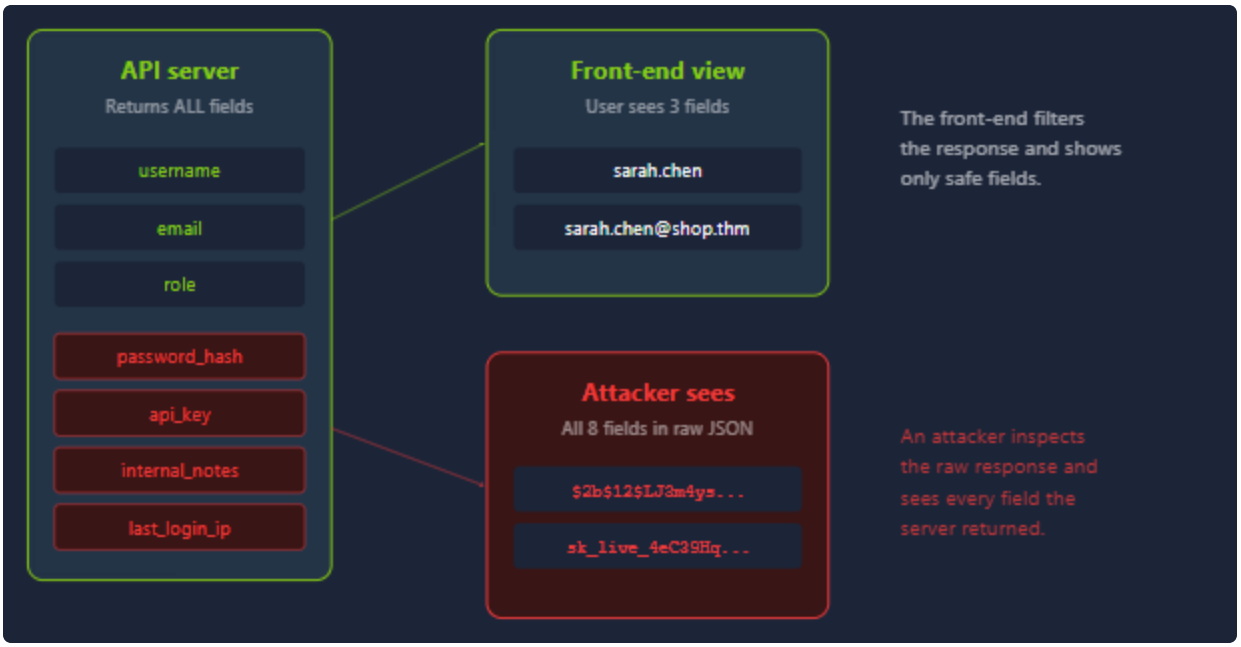
JWT Implementation Flaws

Weak signing secrets are one of the most common JWT issues. JWTs signed with HS256 rely on a secret key. If the secret is guessable (such as `secret`, `password`, or the company name), an attacker can crack it offline with tools like `hashcat` or `jwt_tool` and forge arbitrary tokens.

The `none` algorithm attack exploits APIs that accept unsigned JWTs. An attacker sets `"alg": "none"` in the header, crafts any claims they want, strips the signature, and the server accepts it.

Missing expiration validation means a stolen token remains valid indefinitely if the server does not check the `exp` claim.

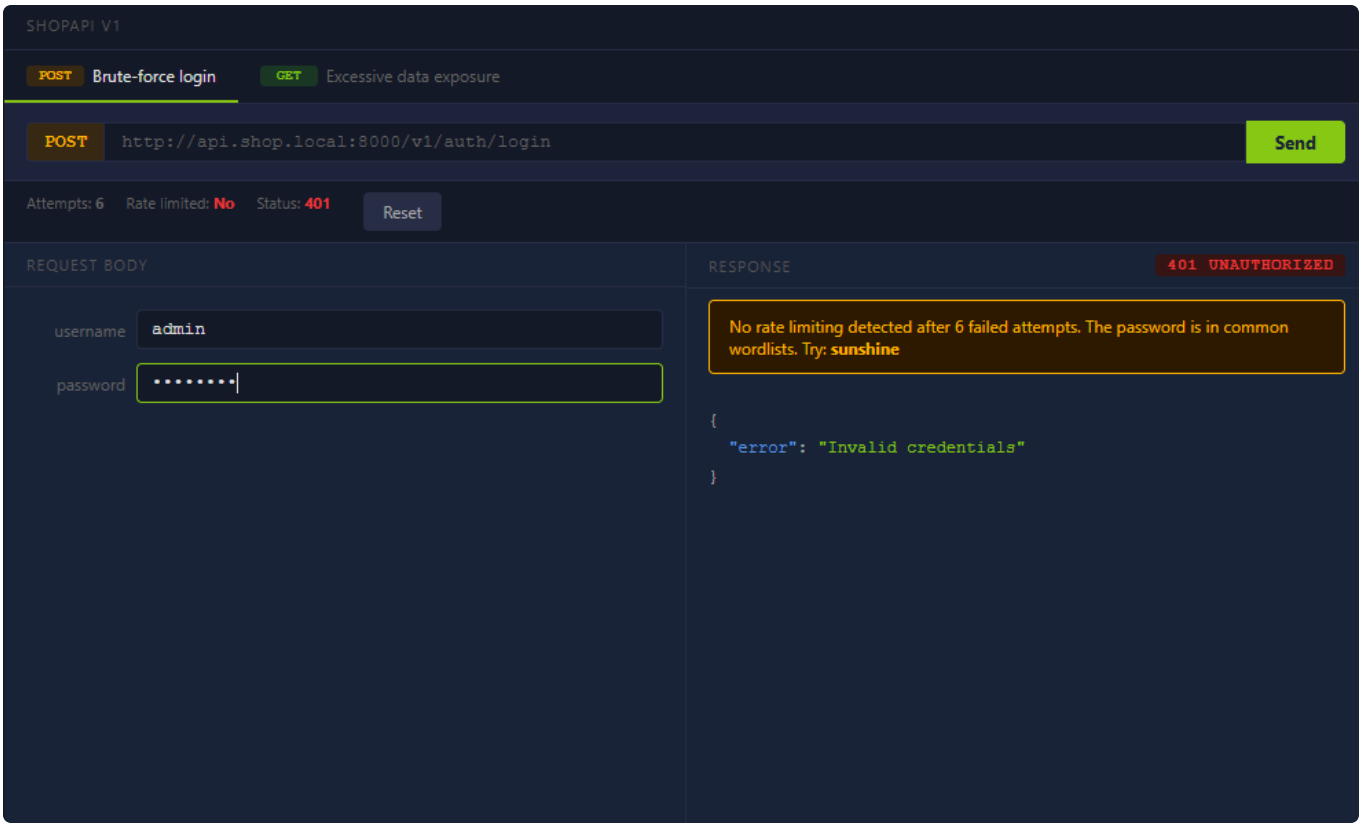
Excessive Data Exposure



API-driven architectures, the back-end returns raw JSON and the front-end decides what to display. The problem arises when developers return entire database objects and rely on the front-end to filter sensitive fields.

The front-end might render `username`, `email`, and an avatar. However, the raw API response may also contain `password_hash`, `api_key`, `internal_notes`, and `last_login_ip`.

practical

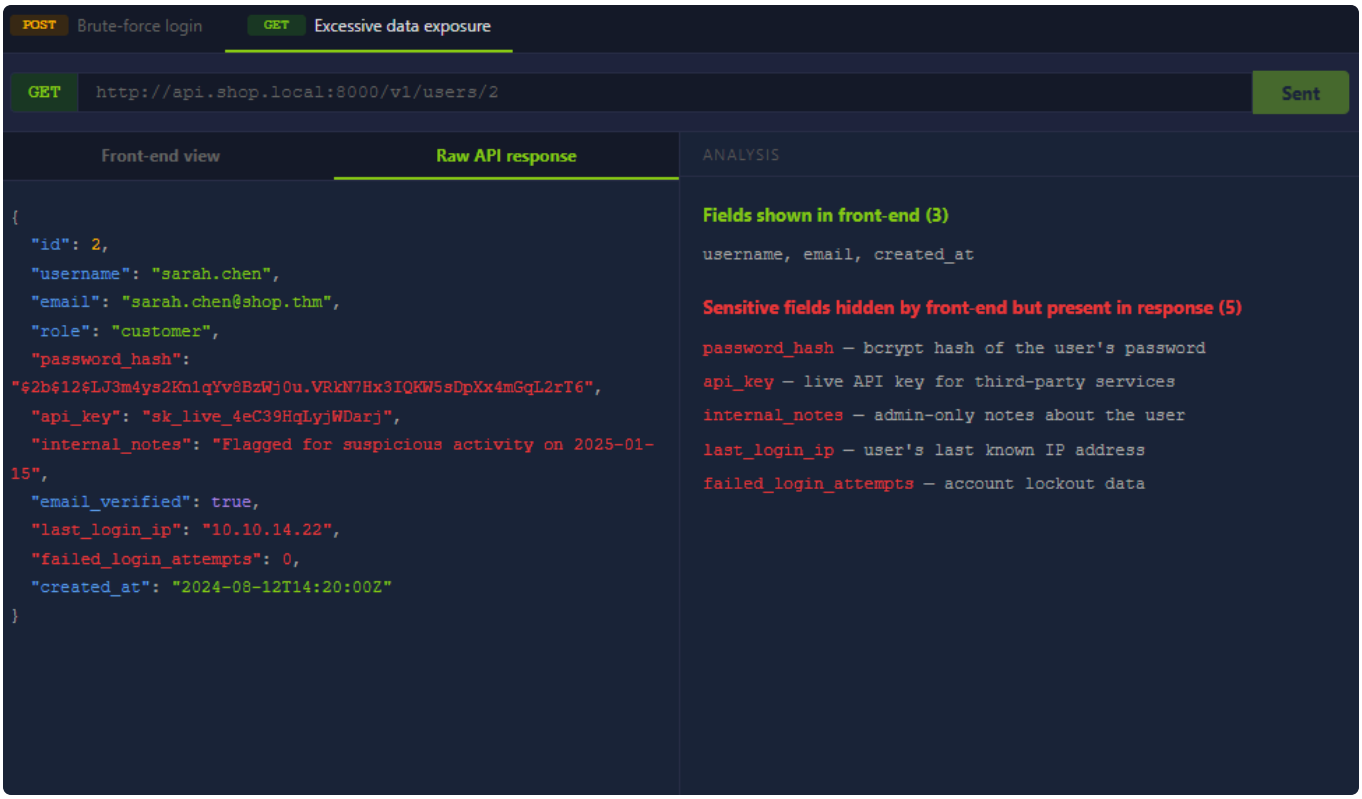


I can keep entering credentials as the API never returns a too many requests error - this means that there is no rate limiting

In a real engagement, you would use a tool like `ffuf` to automate this process against a common password wordlist

Excessive Data Exposure

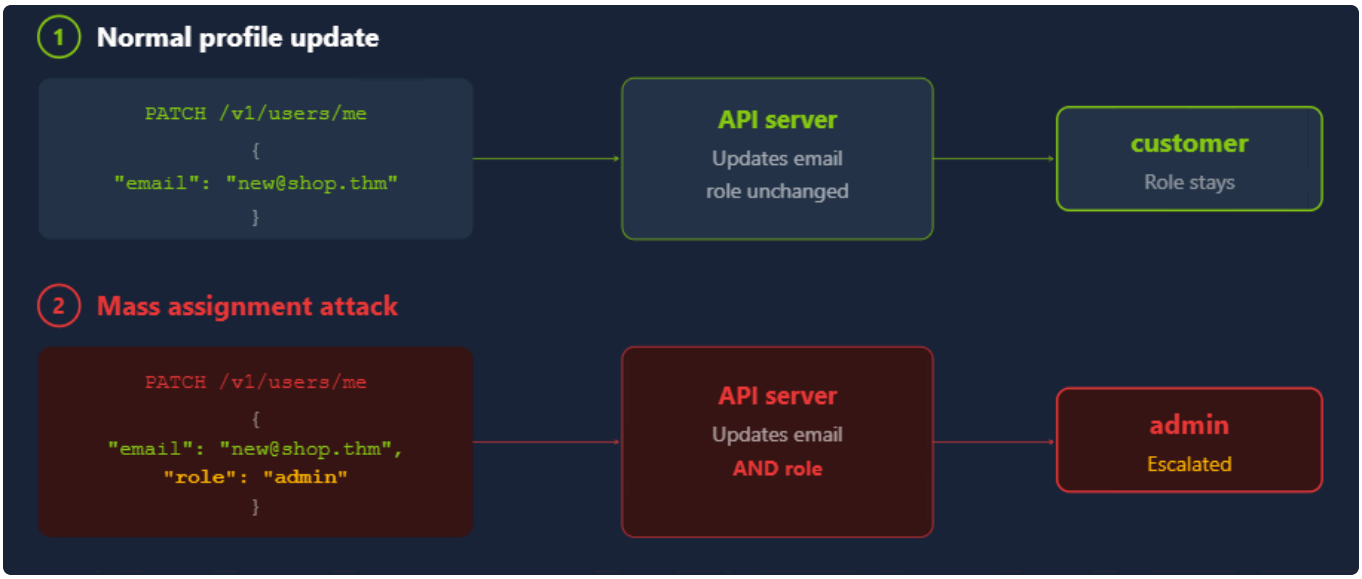
The right panel shows a user profile for sarah.chen. Toggle between **Front-End View** and **Raw API Response**. The front-end view renders only the username, email, and membership date. The raw API response contains `password_hash`, `api_key`, `internal_notes`, and `last_login_ip`. These fields are returned by the server in every response, but the front-end simply hides them.



Mass Assignment and Rate Limiting

Mass assignment is a vulnerability that occurs when an API takes data from a client's request and applies it directly to an internal object without filtering which fields the client is permitted to set.

When a user updates their profile, the front-end sends fields like `email` or `username`. However, the server-side user object likely has additional fields: `role`, `is_admin`, `account_balance`, `email_verified`, and so on. In normal operation, these are set by server-side logic.



If the API does not distinguish between client-writable and server-controlled fields, an attacker can inject additional parameters.

a `PATCH /v1/users/me` request with `{"email": "new@shop.thm", "role": "admin"}` would update both fields if the API blindly accepts everything in the request body

Finding Mass Assignment Opportunities

The most reliable way to discover injectable fields is from the API's own responses. If a user profile response includes `role`, `is_admin`, or `credit_balance`, those field names indicate what attributes exist on the object.

Exposed API documentation is another source. OpenAPI specifications sometimes mark fields as `readOnly`, which hints that the field exists but is intended to be server-controlled.

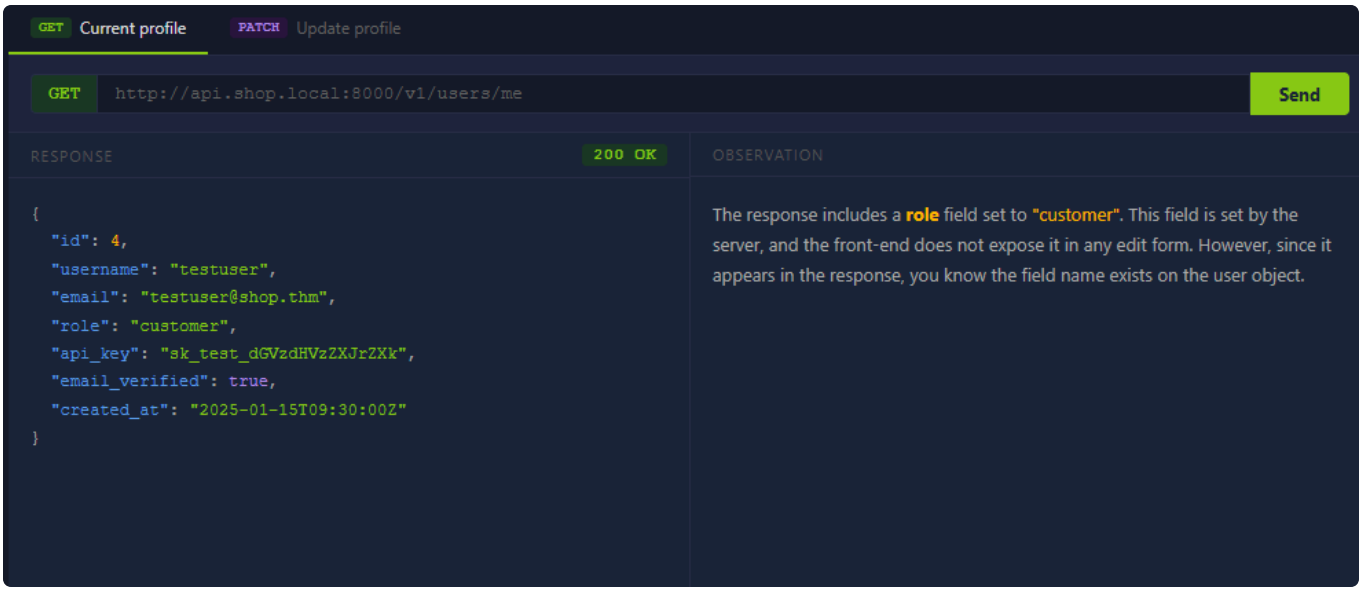
Mass assignment is not limited to profile updates. It can appear in any endpoint that accepts a request body, including registration, order modification, and settings changes.

Rate Limiting Beyond Authentication

Without limits, an attacker can brute-force OTP codes (a 4-digit code has only 10,000 combinations), scrape user data at scale, abuse endpoints that trigger costly actions like SMS or email sends, or overwhelm the API to cause denial of service.

Testing for rate limiting is straightforward: send a burst of identical requests and check whether the API ever returns `429 Too Many Requests`. Well-implemented rate limiting includes headers like `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset`. If these are absent, rate limiting is likely not in place.

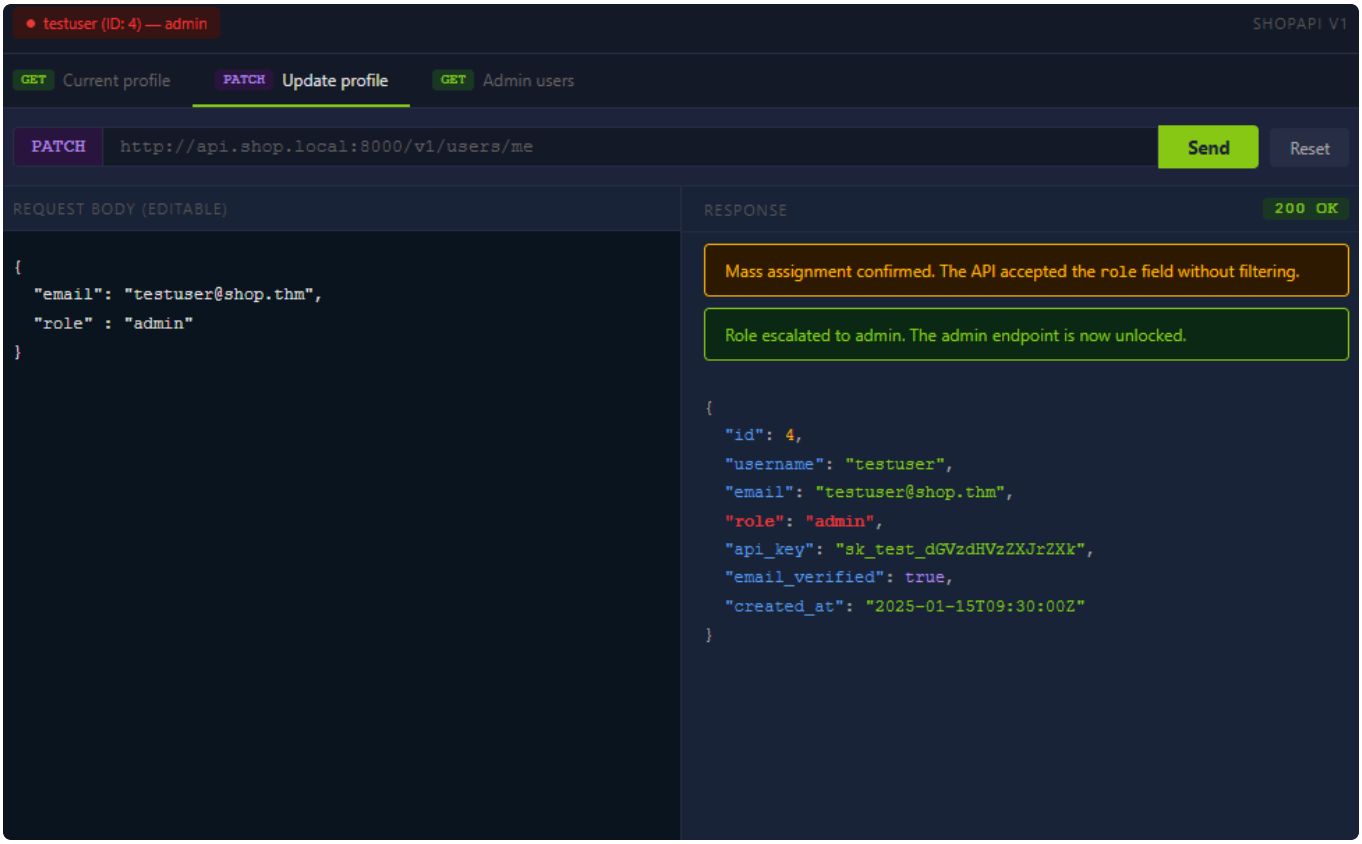
Practical



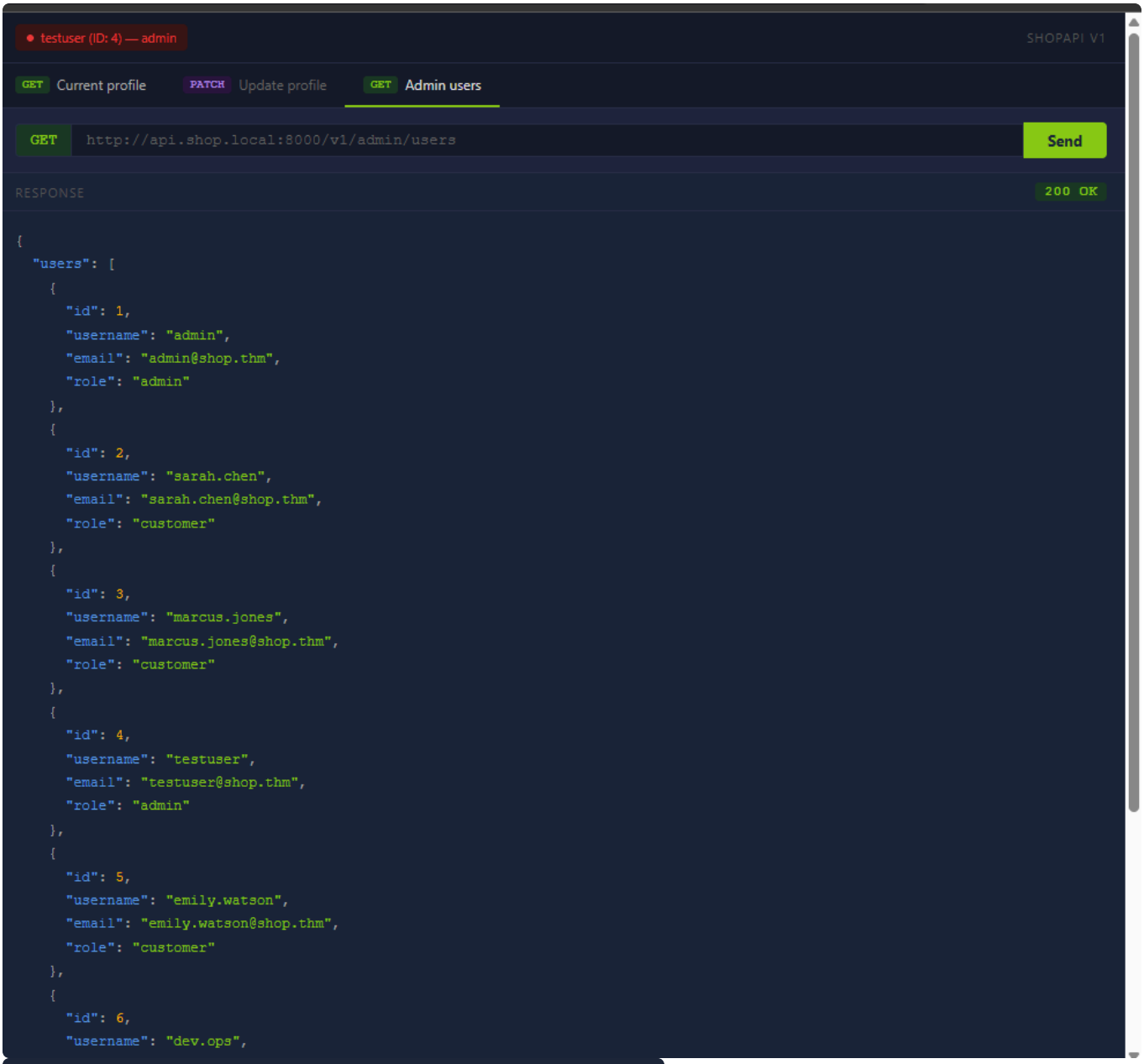
The request exposes the user role even if we are unable to edit this field we know that field name exists

in the patch request:

I added the JSON for the admin role and sent the request and it escalated the role on the account to admin.



This opened up an admin request page and i got all the users and the flag

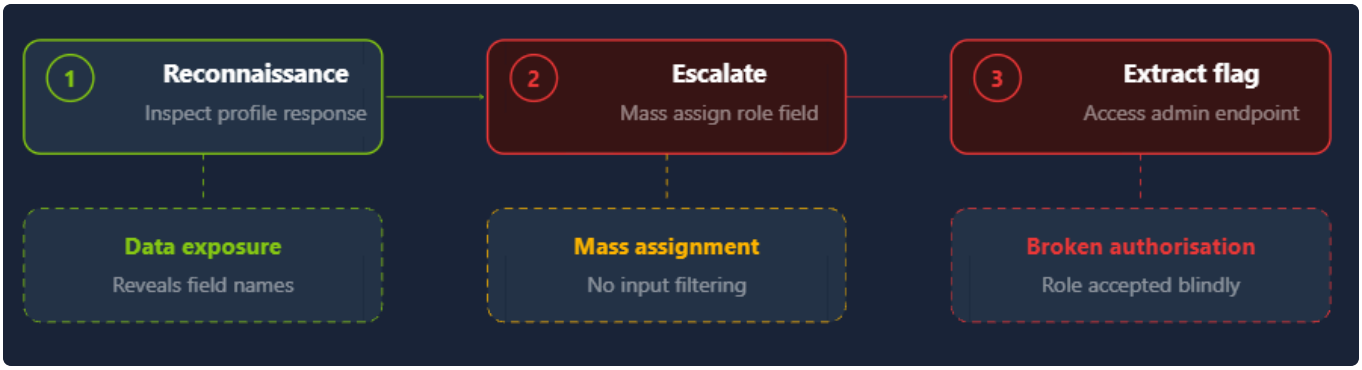



```
    {
      "id": 7,
      "username": "james.miller",
      "email": "james.miller@shop.thm",
      "role": "customer"
    }
  ],
  "total": 7,
  "flag": "THM{m4ss_4ss1gnm3nt_pr1v_3sc}"
}
```

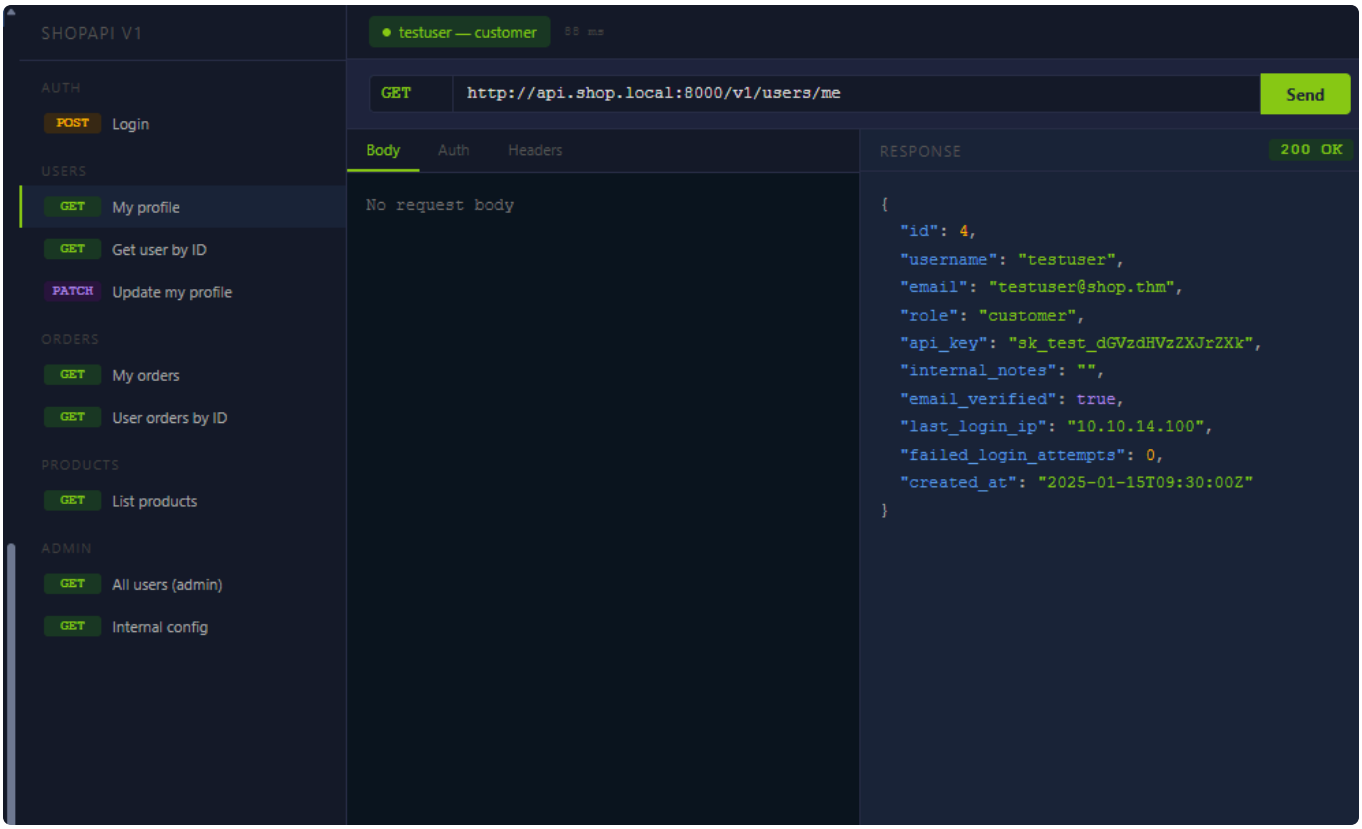

Putting it all together

exploiting an API typically involves chaining multiple findings together. A BOLA vulnerability on its own might provide read access to another user's data. Combined with excessive data

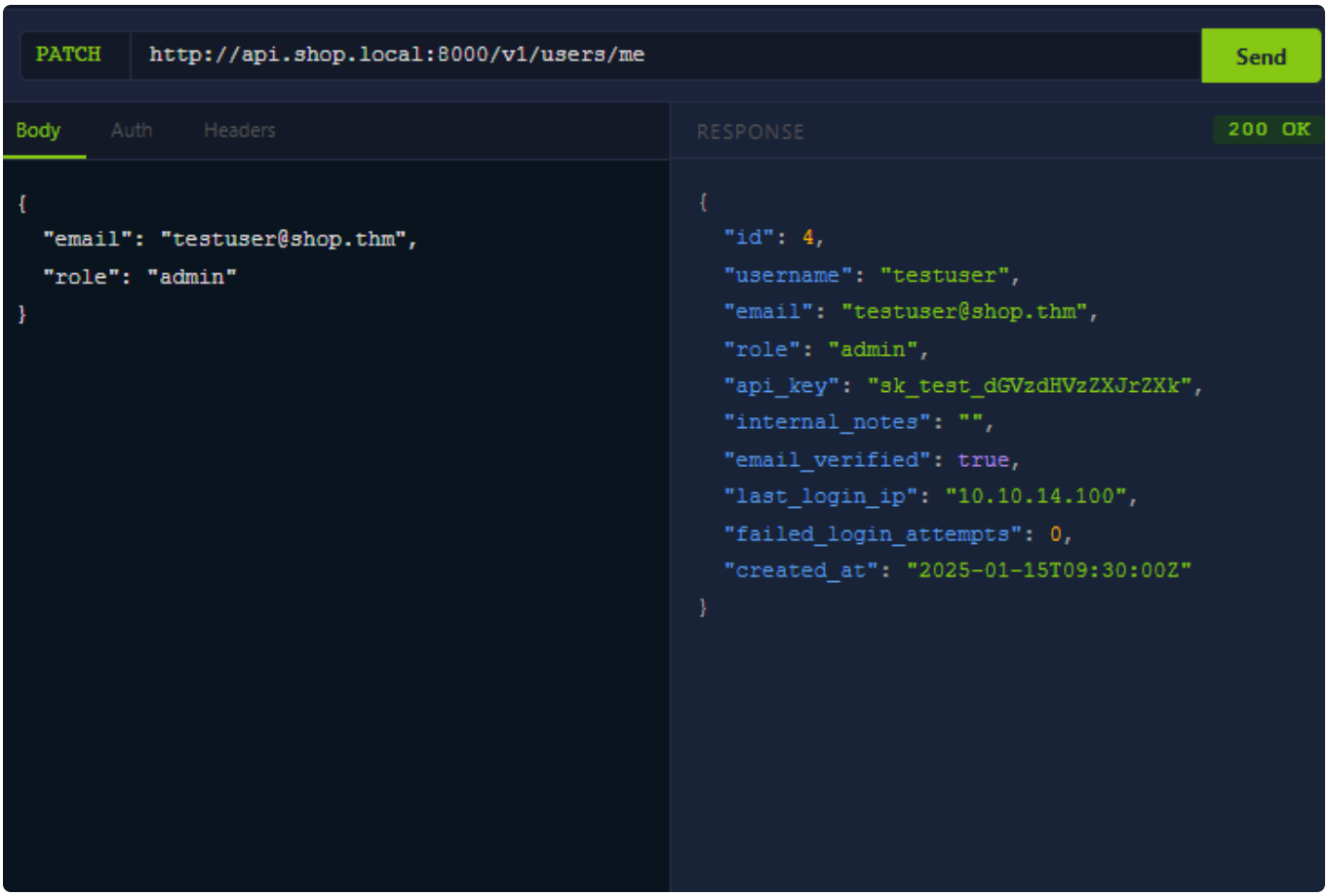
exposure and mass assignment, it provides a path from regular user to full administrative control.



I inspect my own profile with the request and see that there is a role field. my current role is customer this tells me about the fields in the user objects

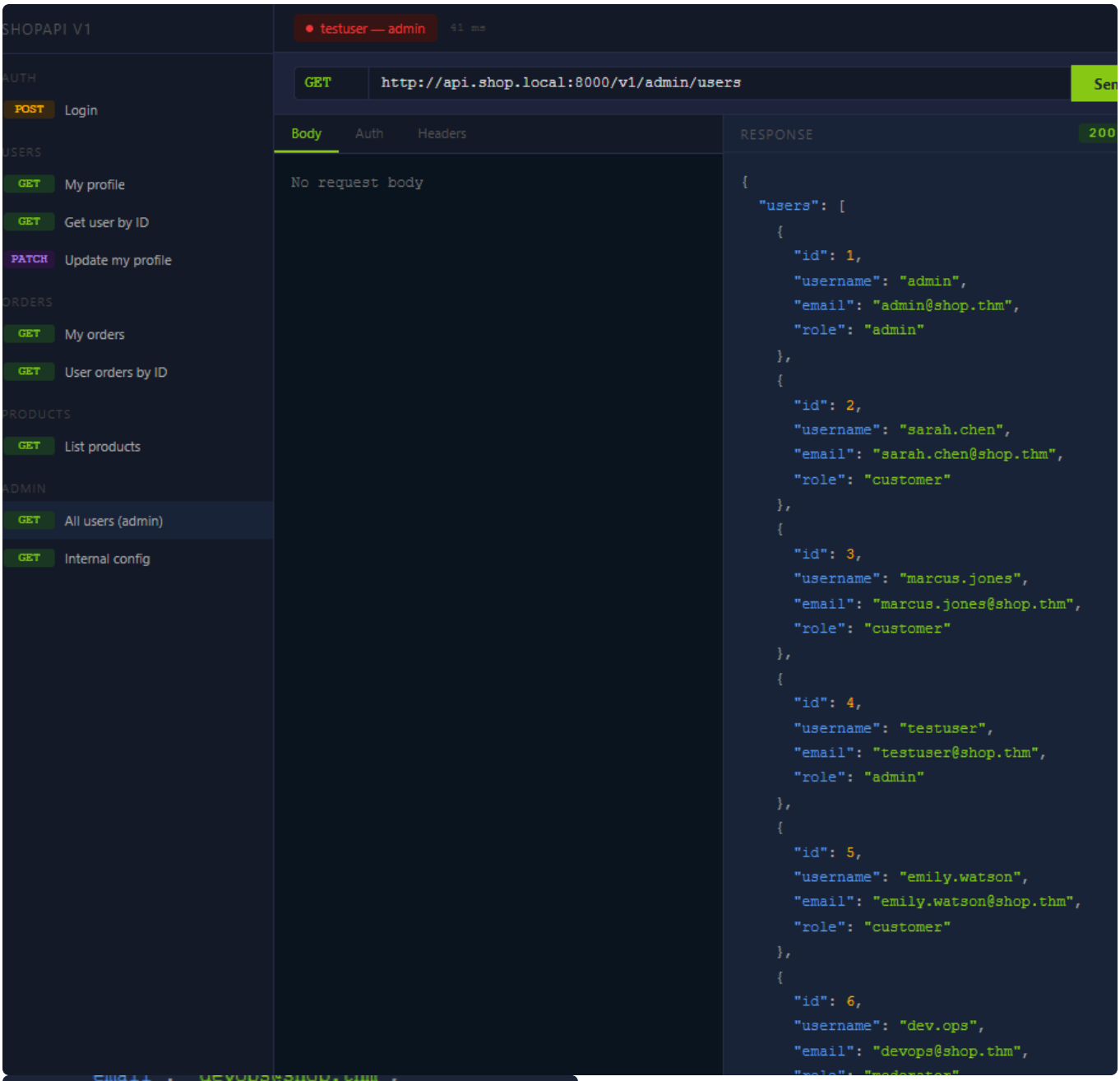


I then navigate to the update my profile and I add the role field and input admin to try and escalate my account role



we can see that its updated my role to admin

then I went onto the admin requests on the left these previously wouldnt open if i wasnt admin but since I edited my role I was able to access these




```
    "role": "moderator"
  },
  {
    "id": 7,
    "username": "james.miller",
    "email": "james.miller@shop.thm",
    "role": "customer"
  }
],
"total": 7,
"flag": "THM{ch41ned_ap1_vulns_f0r_the_w1n}"
```


i accessed all the users and managed to get the flag

very basic flow of what happened:

Step 1 requires you to inspect your own profile and identify a field that reveals information about the user object's structure.

Step 2 requires you to craft a request that modifies a field the front-end does not expose.

Step 3 requires you to use your new access level to query a restricted endpoint.

Conclusion

Defensive Measures

Object-level authorisation: Every endpoint that references a specific resource must verify that the authenticated user is permitted to access it. This check cannot be handled by a global middleware alone, because the relationship between a user and a resource differs for every endpoint.

Response filtering: Return only the fields appropriate for the requesting user's role. Do not serialise entire database objects. Many frameworks provide serialisation libraries or response schema definitions for this purpose.

Input allowlisting: Define an explicit list of writable fields for every endpoint that accepts a request body. Silently ignore anything not on the list. Fields like `role`, `is_admin`, and `permissions` should never be writable through client-facing endpoints.

Rate limiting: Apply globally, with stricter limits on sensitive endpoints. Include standard rate limiting headers (`X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`) in responses.

JWT security: Use strong signing secrets, reject the `none` algorithm, validate `exp` claims, and keep token lifetimes short.

Useful tools for real-world API testing include **Burp Suite** for intercepting API traffic, **Insomnia** and **Postman** for crafting and organising API requests, **ffuf** for endpoint brute-forcing, **jwt_tool** for JWT testing, and **OWASP ZAP** as an open-source alternative.